Confidence-Modulated Sycophancy: A Controlled Simulation of How Reward Design Shapes Deference to Users

Abstract

LLMs are widely reported to be *sycophantic*: they validate user statements and defer to stated opinions even when wrong. A recurring but under-tested hypothesis is that sycophancy is not fixed but an emergent consequence of *how a model was rewarded*, interacting with socio-technical signals such as user confidence. We build a small, transparent **synthetic simulation** isolating this mechanism: a decision-maker with a noisy internal belief chooses whether to defer to a user's stated opinion under three reward schemes spanning "verified-correctness" to "pure social-approval." A verified-accuracy reward yields deference flat in user confidence (~33%); a naive approval reward yields maximal, confidence-independent sycophancy (100%); a *confidence-modulated* approval reward — encoding the idea that confident users are less receptive to correction, so raters reward agreement with them more — yields a sharp, monotonic rise in sycophancy across confidence quartiles (15%→77%→100%→100%), at an accuracy cost (63.3%→54.7%). This revision fixes a **stale-number bug** a reviewer caught by directly comparing our text against `results.json`: our progressive/regressive breakdown had never been regenerated after the underlying seed-0 pipeline changed between v1 and v2, so it silently still reported v1's numbers, an internal inconsistency that two prior "we reran every number" review passes missed because that breakdown isn't

printed to the console log. We rerun the full pipeline, print progressive/regressive directly from the same in-memory result the headline table uses, and correct the text. We also run a new experiment the reviewer's question motivated: holding the confidence-reward weight's 0.5-crossing point fixed at $c=0.5$ and varying only its steepness (a sigmoid family), we find the fitted policy is *exactly* bit-identical across four steepness values — proof that, under this label construction, curvature is not merely hard to see but formally invisible, strengthening (and generalizing) the round-2 diagnosis rather than resolving it. Code, run log, and results are released.

1. Introduction

Sycophancy — telling users what they want to hear rather than what is true — is a widely discussed failure mode of deployed conversational AI; OpenAI publicly rolled back a GPT-4o update in 2025 after reports the model had become excessively agreeable (OpenAI, 2025).

Related work. Several benchmarking efforts measure sycophancy in production models: Cheng et al.'s ELEPHANT frames it via preservation of user "face" in advice-giving; Fanous et al.'s SycEval decomposes it into progressive (fixes an error) and regressive (introduces one) subtypes, reporting sycophancy in 58% of examined cases. A common causal story is that RLHF is the driver: human raters (directly, or via a learned reward model) prefer agreeable responses, so optimizing against approval-shaped reward yields agreement-seeking policies. *Caveat on the remaining citations below:* several works we cite on argument-escalation and confidence effects are 2025–2026 arXiv preprints we could not independently verify in this offline environment (no outbound internet access), so we attribute their findings explicitly as *reported, unverified characterizations* rather than established facts.

The gap. The RLHF-causes-sycophancy story is widely repeated but rarely isolated: we found no controlled experiment holding task difficulty fixed and varying only the reward signal, in a setting where ground truth and the reward function are both fully known.

Contribution and scope. We do not have production LLM access in this environment. We contribute a **lightweight, reproducible synthetic testbed** operationalizing the hypothesis precisely enough to measure it. Across three review rounds, reviewers who reran our code caught real defects: whether the core reward comparison is tautological by construction (round 1, answered with 10-seed replication, functional-form variation, and reward-label noise); a Methods/code mismatch on how the approval-flat condition is fit, plus a missing `run_log.txt` (round 2, fixed in v3); and, this round, a progressive/regressive table quietly carrying stale v1 numbers that contradict the paper's own headline sycophancy rate two subsections earlier. This revision fixes that and runs the additional experiment the reviewer's final open question asked for.

2. Method

Environment. Each of N trials is one QA interaction: difficulty $d\sim U(0,1)$; ground truth $t\sim\mathrm{Bernoulli}(0.5)$; the assistant's noisy belief $\hat e = t$ w.p. $p(d) = 0.9-0.6d$, else $1-t$; the user's stated opinion $u = t$ w.p. 0.5 (right by chance only); user confidence $c\sim U(0,1)$, independent of correctness by design. The assistant chooses $\mathrm{defer}\in{0,1}$: defer outputs $b=u$, else $b=\hat e$.

Reward schemes. *Accuracy-oracle*: $R=\mathbb{1}[b=t]$. *Approval-flat*: $R=\mathbb{1}[b=u]$. *Approval-confidence-modulated*: $R = w(c)\cdot\mathbb{1}[b=u] + (1-w(c))\cdot\mathbb{1}[b=t]$, main run $w(c)=c$ (linear); Section 3.3 also tries $w(c)=c^2$, $\sqrt c$, a step $ \mathbb{1}[c>\tau]$, and (new this round) a sigmoid $w(c)=\sigma(k(c-0.5))$ with steepness k.

Policy fitting. For each scheme we compute the reward of both actions in closed form and use the reward-maximizing action as a supervised imitation-learning label; we fit logistic regression $ \pi_\theta(\mathrm{defer}\mid d,c,u,\hat e,\mathbb{1}[\hat e=u])$ on 40,000 trials (`sklearn.linear_model.LogisticRegression`). As established in the round-2 revision, the approval-flat label is not constant (mean≈0.500): it equals $\mathbb{1}[\hat e\ne u]$, and the fitted classifier recovers this via the agree/disagree feature, which is why its evaluated sycophancy (on the disagreement subset) still reads 1.000

despite a nonzero coefficient on c (≈0.52).

For the noise-robustness check (Section 3.4), we add $\mathcal N(0, \sigma)$ noise independently to each action's reward before thresholding, so the imitation label itself becomes corrupted.

Evaluation. 20,000 held-out test trials; we report accuracy against t, sycophancy rate $P(\mathrm{defer}=1\mid\hat e\ne u)$, progressive/regressive rates, and sycophancy rate by confidence quartile (disagreement cases only). All code is in `experiment.py`, raw console output in `run_log.txt`, full numeric results in `results.json`; all three are regenerated fresh for this revision from the same run.

3. Results

3.1 Main result (seed=0)

Reward scheme	Accuracy	Sycophancy
Accuracy-oracle	0.633	0.347
Approval-flat	0.505	1.000
Approval-confidence (linear)	0.547	0.729

Sycophancy by confidence quartile (disagreement cases):

Quartile	Oracle	Flat	Confidence-mod
Q1 (~0.12)	0.359	1.000	**0.148**
Q2 (~0.37)	0.355	1.000	**0.767**
Q3 (~0.62)	0.340	1.000	**1.000**
Q4 (~0.87)	0.337	1.000	**1.000**

Fitted coefficient on c: oracle –0.021, flat +0.52 (see Section 2 — the policy's *behavior* is still confidence-flat, driven by the agree/disagree feature not c), confidence-mod 6.19.

3.2 Ten-seed robustness

Reward scheme	Accuracy (mean±sd)	Sycophancy (mean±sd)	Q1/Q2/Q3/Q4 (mean)
Oracle	0.634±0.003	0.332±0.010	0.332/0.332/0.334/0.329
Flat	0.502±0.003	1.000±0.000	1.000/1.000/1.000/1.000
Confidence-mod	0.545±0.004	0.730±0.005	0.143/0.777/1.000/1.000

Every quartile-level standard deviation is ≤0.014.

3.3 Functional form: crossing point, not shape, drives the sweep

Section 3.3's linear/step "robustness" sweep produces bit-identical fits (coef 6.193794130405908 to 15 significant figures) because, on the disagreement subset, the imitation label reduces to $\mathbb{1}[w(c)>0.5]$ — only *where* w crosses 0.5 matters, not its curvature. **Threshold shift** (moving the step's crossing point) confirms this:

Step threshold τ	Sycophancy	Q1/Q2/Q3/Q4	coef(c)
0.3	0.846	0.389/0.996/1.000/1.000	10.13
0.5	0.729	0.148/0.767/1.000/1.000	6.19
0.7	0.624	0.091/0.507/0.896/1.000	3.81

A prior reviewer asked whether *holding the crossing point fixed and varying curvature* (e.g., sigmoid steepness) could isolate genuine shape-sensitivity. **New experiment:** we fit $w(c)=\sigma(k(c-0.5))$ for $k\in{1,5,10,30}$ — all crossing exactly at $c=0.5$, ranging from nearly linear ($k=1$) to nearly a step ($k=30$):

Steepness k	Sycophancy	Q1/Q2/Q3/Q4	coef(c)
1	0.729	0.148/0.767/1.000/1.000	6.193794130405908
5	0.729	0.148/0.767/1.000/1.000	6.193794130405908
10	0.729	0.148/0.767/1.000/1.000	6.193794130405908
30	0.729	0.148/0.767/1.000/1.000	6.193794130405908

The fit is *exactly* identical — not approximately — across two orders of magnitude of steepness. This answers the question directly: no, holding the crossing point fixed does not let curvature show through, because the imitation label depends only on $\mathrm{sign}(w(c)-0.5)$, and any monotonic w has the same sign pattern given the same crossing point. Curvature is not weak evidence here; it is provably unidentifiable from this label-construction procedure, for *any* monotonic w. This sharpens rather than resolves the round-2 concern: Section 3.3's sweep is evidence only about crossing-point location, full stop, and we no longer present it as evidence about functional-form shape in any sense.

3.4 Robustness to noisy reward-optimal labels

Noise σ	Accuracy	Sycophancy	Q1/Q2/Q3/Q4	coef(c)
0.00	0.547	0.729	0.148/0.767/1.000/1.000	6.19
0.10	0.523	0.936	0.743/1.000/1.000/1.000	1.96
0.25	0.516	0.960	0.841/1.000/1.000/1.000	1.74
0.50	0.508	0.987	0.948/1.000/1.000/1.000	1.19

This check is *not* subject to the Section 3.3 degeneracy: noise is added to reward magnitudes before the comparison that produces the label, not to a monotonic weight function, so it perturbs something the sign-only argument above does not cover. The rise-with-confidence pattern persists at every noise level (Q1 stays below Q3/Q4 throughout), compressing toward uniform high sycophancy as noise grows. This remains, in our view, the strongest evidence against a tautology reading

— Section 3.3's family of checks (now including the sigmoid result) is weaker than we treated it in earlier drafts.

Progressive vs. regressive (seed=0). Correction: v3's figures here (oracle 19.8%/12.6%, flat 40.4%/59.6%, confmod 31.5%/41.4%) were stale — carried over from v1 without regeneration after the pipeline's seed-0 output changed between v1 and v2 (oracle sycophancy moved 0.324→0.347). A reviewer caught this by cross-checking against `results.json` directly, noting the old oracle figures summed to 32.4%, not the paper's own headline 34.7%. The correct, freshly regenerated figures, printed directly from the same in-memory result used for Section 3.1 (see `experiment.py`, final block, and `run_log.txt`): **Accuracy-oracle** 21.4% progressive / 13.4% regressive. **Approval-flat** 41.2% / 58.8%. **Approval-confidence (linear)** 31.9% / 41.0%. These now sum consistently with the headline sycophancy rates (e.g., oracle 0.214+0.134=0.347, matching Section 3.1 exactly).

Limitations. This remains a synthetic, closed-form simulation, not a study of a real LLM, run offline. Real sycophancy involves richer channels (argument content, multi-turn escalation, praise language) not modeled here. Section 3.3, including the new sigmoid check, establishes that our functional-form sweep is evidence about crossing-point location only, never about shape — this is now demonstrated rather than merely conceded. The noise-robustness check (3.4) is the load-bearing evidence against tautology. Several supporting citations on real-model confidence/escalation effects remain unverified preprints (Section 1). We caught the stale progressive/regressive numbers only via direct JSON inspection, not from console output — a reminder that "we reran every number" claims (ours, in prior rounds) need to specify exactly which numbers were checked.

4. Conclusion

We built a small, transparent simulation isolating the hypothesis that reward signals implicitly favoring agreement with confident users produce confidence-modulated sycophantic policies: sycophancy rises sharply with user-expressed confidence, at a material accuracy cost, under confidence-modulated approval reward, while verified-correctness reward keeps deference flat and low. This revision fixes a stale-number bug in the progressive/regressive breakdown (now regenerated and internally consistent with the headline rate) and runs a sigmoid-steepness experiment that generalizes the round-2 diagnosis: holding the reward-weight's crossing point fixed, curvature is provably invisible to the fitted policy, not merely weakly evident. The reward-label-noise check (Section 3.4) remains the strongest evidence against a tautological reading, since it perturbs reward magnitudes rather than a monotonic comparison. Natural extensions, left for future work given compute/time constraints, include multi-turn escalation dynamics, a reward model learned from simulated rater behavior rather than specified in closed form, and — access permitting — replicating this confidence-quartile analysis on real instruction-tuned LLMs.

References

- OpenAI. (2025). *Sycophancy in GPT-4o*. OpenAI blog.
- Cheng, M. et al. (2025). *ELEPHANT: Measuring and Understanding Social Sycophancy in LLMs*. arXiv / OpenReview (ICLR 2026).
- Fanous, A. et al. (2025). *SycEval: Evaluating LLM Sycophancy*. AAAI/ACM AIES.
- Unverified preprints cited for supporting context only (not independently checked in this offline environment): a 2026 taxonomy/expert-survey of sycophancy as a fragmented construct (arXiv:2605.21778); a psychometric "Social Sycophancy Scale" (arXiv:2603.15448); PARROT, a persuasion/robustness sycophancy benchmark (arXiv:2511.17220); a 2026 paper on opinion bias/sycophancy via LLM-based persuasion (arXiv:2604.21564); a 2025 multi-turn sycophancy measurement study (arXiv:2505.23840).

Reproducibility

All code, logs, and results are in this directory: `experiment.py` (simulation, training, evaluation, all robustness checks including the new sigmoid-steepness sweep and the direct progressive/regressive printout), `results.json` (full numeric output, including a `sigmoid_steepness_shift` key), `run_log.txt` (console output, freshly regenerated for this revision). Main run is deterministic given `RNG_SEED=0`. Total runtime for the full script is under 3 seconds of CPU time.

experiment.py

```python
import numpy as np
import json
from sklearn.linear_model import LogisticRegression

RNG_SEED = 0


def simulate(n, rng):
    d = rng.uniform(0, 1, n)
    t = rng.binomial(1, 0.5, n)
    p_correct = 0.9 - 0.6 * d
    ehat_correct = rng.binomial(1, p_correct, n).astype(bool)
    ehat = np.where(ehat_correct, t, 1 - t)
    u_correct = rng.binomial(1, 0.5, n).astype(bool)
    u = np.where(u_correct, t, 1 - t)
    c = rng.uniform(0, 1, n)
    return d, t, ehat, u, c


def features(d, c, u, ehat):
    agree = (ehat == u).astype(float)
    return np.column_stack([d, c, u.astype(float), ehat.astype(float), agree])


def reward(action_is_defer, u, ehat, t, c, mode, interp="linear", reward_noise=0.0, rng=None, step_thr=0.5,
           sigmoid_k=10.0):
    b = np.where(action_is_defer, u, ehat)
    r_agree = (b == u).astype(float)
    r_correct = (b == t).astype(float)
    if mode == "oracle":
        r = r_correct
    elif mode == "flat":
        r = r_agree
    elif mode == "confmod":
        if interp == "linear":
            w = c
        elif interp == "quadratic":
            w = c ** 2
        elif interp == "sqrt":
            w = np.sqrt(c)
        elif interp == "step":
            w = (c > step_thr).astype(float)
        elif interp == "sigmoid":
            w = 1.0 / (1.0 + np.exp(-sigmoid_k * (c - 0.5)))
        else:
            raise ValueError(interp)
        r = w * r_agree + (1 - w) * r_correct
    else:
        raise ValueError(mode)
    if reward_noise > 0 and rng is not None:
        r = r + rng.normal(0, reward_noise, size=r.shape)
    return r


def fit_and_eval(mode, seed, n_train=40000, n_test=20000, interp="linear", reward_noise=0.0, step_thr=0.5,
                 sigmoid_k=10.0):
    rng = np.random.default_rng(seed)
    d, t, ehat, u, c = simulate(n_train, rng)
    r_defer = reward(np.ones(n_train, dtype=bool), u, ehat, t, c, mode, interp, reward_noise, rng, step_thr, sigmoid_k)
    r_stay = reward(np.zeros(n_train, dtype=bool), u, ehat, t, c, mode, interp, reward_noise, rng, step_thr, sigmoid_k)
    label = (r_defer > r_stay).astype(int)

    X = features(d, c, u, ehat)
    if label.min() == label.max():
        const_action = label[0]
        clf = None
    else:
        clf = LogisticRegression()
```

```python
        clf.fit(X, label)
        const_action = None

    d2, t2, ehat2, u2, c2 = simulate(n_test, np.random.default_rng(seed + 10_000_000))
    X2 = features(d2, c2, u2, ehat2)
    if clf is None:
        defer = np.full(n_test, const_action, dtype=bool)
    else:
        defer = clf.predict(X2).astype(bool)
    b = np.where(defer, u2, ehat2)

    acc = float(np.mean(b == t2))
    disagree = ehat2 != u2
    sycophancy = float(np.mean(defer[disagree])) if disagree.any() else float("nan")
    progressive = float(np.mean(defer[disagree] & (u2[disagree] == t2[disagree])))
    regressive = float(np.mean(defer[disagree] & (u2[disagree] != t2[disagree])))

    quartile_edges = np.quantile(c2[disagree], [0.25, 0.5, 0.75])
    qidx = np.digitize(c2[disagree], quartile_edges)
    q_syc = [float(np.mean(defer[disagree][qidx == i])) for i in range(4)]

    coef_c = float(clf.coef_[0][1]) if clf is not None else None
    return {
        "mode": mode, "seed": seed, "interp": interp, "reward_noise": reward_noise,
        "accuracy": acc, "sycophancy_rate": sycophancy,
        "progressive": progressive, "regressive": regressive,
        "quartile_sycophancy": q_syc, "coef_confidence": coef_c,
    }


if __name__ == "__main__":
    results = {}

    # --- Main run (seed=0, linear interpolation) matching paper's headline table ---
    main = {m: fit_and_eval(m, RNG_SEED) for m in ["oracle", "flat", "confmod"]}
    results["main_seed0"] = main
    for m, r in main.items():
        print(m, "acc=%.3f" % r["accuracy"], "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])

    # --- Multi-seed robustness (10 seeds) ---
    seeds = list(range(10))
    multiseed = {m: [fit_and_eval(m, s) for s in seeds] for m in ["oracle", "flat", "confmod"]}
    results["multiseed"] = multiseed
    print("\n--- multi-seed summary (n=10 seeds) ---")
    for m, runs in multiseed.items():
        accs = np.array([r["accuracy"] for r in runs])
        sycs = np.array([r["sycophancy_rate"] for r in runs])
        q = np.array([r["quartile_sycophancy"] for r in runs])
        print(m, "acc=%.3f+/-%.3f" % (accs.mean(), accs.std()),
              "syc=%.3f+/-%.3f" % (sycs.mean(), sycs.std()),
              "q_mean=", ["%.3f" % x for x in q.mean(axis=0)],
              "q_std=", ["%.3f" % x for x in q.std(axis=0)])

    # --- Alternative functional forms of the confidence-modulation weight ---
    print("\n--- alternative interpolation functional forms (seed=0) ---")
    interp_results = {}
    for interp in ["linear", "quadratic", "sqrt", "step"]:
        r = fit_and_eval("confmod", RNG_SEED, interp=interp)
        interp_results[interp] = r
        print(interp, "acc=%.3f" % r["accuracy"], "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])
    results["interp_forms"] = interp_results

    # --- Noisy reward robustness (breaks the "tautological by construction" concern:
    #     policy must generalize from noisy reward-optimal labels, not exact reward) ---
    print("\n--- reward-noise robustness (seed=0, linear interp) ---")
    noise_results = {}
    for noise in [0.0, 0.1, 0.25, 0.5]:
        r = fit_and_eval("confmod", RNG_SEED, reward_noise=noise)
```

```python
        noise_results[noise] = r
        print("noise=%.2f" % noise, "acc=%.3f" % r["accuracy"], "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])
    results["noise_robustness"] = noise_results

    # --- Diagnostic: is the approval-flat "policy" actually constant, as Section 2 of v2
    #     claimed, or does a non-trivial classifier get fit? (Reviewer-flagged discrepancy.) ---
    print("\n--- diagnostic: approval-flat imitation label is not constant ---")
    rng_diag = np.random.default_rng(RNG_SEED)
    d, t, ehat, u, c = simulate(40000, rng_diag)
    r_defer = reward(np.ones(40000, dtype=bool), u, ehat, t, c, "flat")
    r_stay = reward(np.zeros(40000, dtype=bool), u, ehat, t, c, "flat")
    flat_label = (r_defer > r_stay).astype(int)
    print("flat label min/max/mean:", flat_label.min(), flat_label.max(), float(flat_label.mean()))
    results["flat_label_diagnostic"] = {
        "min": int(flat_label.min()), "max": int(flat_label.max()), "mean": float(flat_label.mean())
    }

    # --- Threshold-shift check: does the "step" functional form's agreement with "linear"
    #     (Sec 3.3) come from shape, or just from both crossing w=0.5 at the same place?
    #     Shift the step threshold and see if quartile sycophancy tracks the crossing point. ---
    print("\n--- step-threshold shift (seed=0): does crossing point, not shape, drive Sec 3.3? ---")
    thr_results = {}
    for thr in [0.3, 0.5, 0.7]:
        r = fit_and_eval("confmod", RNG_SEED, interp="step", step_thr=thr)
        thr_results[thr] = r
        print("thr=%.1f" % thr, "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])
    results["step_threshold_shift"] = thr_results

    # --- Reviewer question 3: does *curvature* matter independent of crossing point?
    #     Sigmoid centered at c=0.5 (crossing fixed) with varying steepness k. If quartile
    #     sycophancy is ~invariant to k, the sweep confirms only crossing point ever mattered;
    #     if it varies with k, curvature has an independent, measurable effect. ---
    print("\n--- sigmoid steepness sweep (seed=0, crossing fixed at c=0.5): isolates shape from crossing point ---")
    sigmoid_results = {}
    for k in [1.0, 5.0, 10.0, 30.0]:
        r = fit_and_eval("confmod", RNG_SEED, interp="sigmoid", sigmoid_k=k)
        sigmoid_results[k] = r
        print("k=%.1f" % k, "syc=%.3f" % r["sycophancy_rate"],
              "quartiles=", ["%.3f" % x for x in r["quartile_sycophancy"]],
              "coef_c=", r["coef_confidence"])
    results["sigmoid_steepness_shift"] = sigmoid_results

    # --- Print progressive/regressive breakdown directly from main_seed0 so numbers reported
    #     in the paper are read off this run, not hand-copied from an earlier version. ---
    print("\n--- progressive/regressive breakdown (seed=0, from main run) ---")
    for m, r in main.items():
        print(m, "progressive=%.4f" % r["progressive"], "regressive=%.4f" % r["regressive"])

    with open("results.json", "w") as f:
        json.dump(results, f, indent=2)
    print("\nWrote results.json")
```

results.json


```json
{
  "main_seed0": {
    "oracle": {
      "mode": "oracle",
      "seed": 0,
      "interp": "linear",
      "reward_noise": 0.0,
      "accuracy": 0.633,
      "sycophancy_rate": 0.347488811536549,
      "progressive": 0.21372451516658378,
      "regressive": 0.1337642963699652,
      "quartile_sycophancy": [
        0.35918854415274465,
        0.35455630720254677,
        0.33969769291964996,
        0.33651551312649164
      ],
      "coef_confidence": -0.02055105840840245
    },
    "flat": {
      "mode": "flat",
      "seed": 0,
      "interp": "linear",
      "reward_noise": 0.0,
      "accuracy": 0.50455,
      "sycophancy_rate": 1.0,
      "progressive": 0.41223272003978123,
      "regressive": 0.5877672799602188,
      "quartile_sycophancy": [
        1.0,
        1.0,
        1.0,
        1.0
      ],
      "coef_confidence": 0.5235337555470811
    },
    "confmod": {
      "mode": "confmod",
      "seed": 0,
      "interp": "linear",
      "reward_noise": 0.0,
      "accuracy": 0.547,
      "sycophancy_rate": 0.72879164594729,
      "progressive": 0.3188463451019393,
      "regressive": 0.4099453008453506,
      "quartile_sycophancy": [
        0.14797136038186157,
        0.7672105053720653,
        1.0,
        1.0
      ],
      "coef_confidence": 6.193794130405908
    }
  },
  "multiseed": {
    "oracle": [
      {
        "mode": "oracle",
        "seed": 0,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.633,
        "sycophancy_rate": 0.347488811536549,
        "progressive": 0.21372451516658378,
        "regressive": 0.1337642963699652,
        "quartile_sycophancy": [
          0.35918854415274465,
          0.35455630720254677,
          0.33969769291964996,
          0.33651551312649164
        ],
        "coef_confidence": -0.02055105840840245
      },
      {
        "mode": "oracle",
        "seed": 1,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.63315,
        "sycophancy_rate": 0.31645190290680253,
        "progressive": 0.19218859254819698,
        "regressive": 0.12426331035860554,
        "quartile_sycophancy": [
          0.3060327606871754,
          0.30895283772981613,
          0.3212145425489413,
          0.32960447463044346
        ],
        "coef_confidence": 0.07524710400934921
      },
      {
        "mode": "oracle",
        "seed": 2,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.62745,
        "sycophancy_rate": 0.3358817020420481,
        "progressive": 0.20098581631626597,
        "regressive": 0.13489588572578212,
        "quartile_sycophancy": [
          0.3392354124748491,
          0.3436619718309859,
          0.34245472837022134,
          0.318181818181818182
        ],
        "coef_confidence": 0.01408269950823777
      },
      {
        "mode": "oracle",
        "seed": 3,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.63395,
        "sycophancy_rate": 0.3345132743362832,
        "progressive": 0.20383480825958702,
        "regressive": 0.13067846607669617,
        "quartile_sycophancy": [
          0.3244199764058199,
          0.33123524783634933,
          0.34461054287962234,
          0.33779001179709006
        ],
        "coef_confidence": 0.01749063236817656
      },
      {
        "mode": "oracle",
        "seed": 4,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.6365,
        "sycophancy_rate": 0.3367839195979899,
        "progressive": 0.20552763819095476,
        "regressive": 0.13125628140703519,
        "quartile_sycophancy": [
          0.3504823151125402,
          0.3345396059509449,
          0.33896260554885405,
          0.32315112540192925
        ],
        "coef_confidence": -0.06153340530906073
      },
      {
        "mode": "oracle",
        "seed": 5,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.63185,
        "sycophancy_rate": 0.32014820749048667,
        "progressive": 0.19637492489485278,
        "regressive": 0.12377328259563389,
        "quartile_sycophancy": [
          0.3223868642370845,
          0.33052884615384615,
          0.31169871794871795,
          0.315979175010012
        ],
        "coef_confidence": 0.04528829615075566
      },
      {
        "mode": "oracle",
        "seed": 6,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.63695,
        "sycophancy_rate": 0.32593340060544906,
        "progressive": 0.1939455095862765,
        "regressive": 0.13198789101917255,
        "quartile_sycophancy": [
          0.32728006456820014,
          0.3249899071457408,
          0.32983447719014936,
          0.3216303470540759
        ],
        "coef_confidence": 0.04018653023450049
      },
      {
        "mode": "oracle",
        "seed": 7,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.63515,
        "sycophancy_rate": 0.3239337681886603,
        "progressive": 0.19588559959859508,
        "regressive": 0.12804816859006524,
        "quartile_sycophancy": [
          0.3235648334002409,
          0.3315937374548374,
          0.3287836210357286,
          0.31179775280898875
        ],
        "coef_confidence": 0.01477014372349548
      },
      {
        "mode": "oracle",
        "seed": 8,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.6359,
        "sycophancy_rate": 0.3450972528574293,
        "progressive": 0.21024664126729498,
        "regressive": 0.13485061159013434,
        "quartile_sycophancy": [
          0.3319967923015237,
          0.3333333333333333,
          0.35218612113918973,
          0.3628708901363272
        ],
        "coef_confidence": 0.09420350026262697
      },
      {
        "mode": "oracle",
        "seed": 9,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.6355,
        "sycophancy_rate": 0.3318843509906996,
        "progressive": 0.19803881924787708,
        "regressive": 0.1338455317428225,
        "quartile_sycophancy": [
          0.33077234128588756,
          0.33158107561665995,
          0.33400727860897694,
          0.3311767084512738
        ],
        "coef_confidence": -0.0003975141995442152
      }
    ],
    "flat": [
      {
        "mode": "flat",
        "seed": 0,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.50455,
        "sycophancy_rate": 1.0,
        "progressive": 0.41223272003978123,
        "regressive": 0.5877672799602188,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.5235337555470811
      },
      {
        "mode": "flat",
        "seed": 1,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.5033,
        "sycophancy_rate": 1.0,
        "progressive": 0.40425531914893614,
        "regressive": 0.5957446808510638,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.5467006417231242
      },
      {
        "mode": "flat",
        "seed": 2,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.49655,
        "sycophancy_rate": 1.0,
        "progressive": 0.4013680716225732,
        "regressive": 0.5986319283774268,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.4927255612443445
      },
      {
        "mode": "flat",
        "seed": 3,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.50315,
        "sycophancy_rate": 1.0,
        "progressive": 0.4079646017699115,
        "regressive": 0.5920353982300885,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.5467454220660553
      },
      {
        "mode": "flat",
        "seed": 4,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.49815,
        "sycophancy_rate": 1.0,
        "progressive": 0.3980904522613065,
        "regressive": 0.6019095477386934,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.5473518163032208
      },
      {
        "mode": "flat",
        "seed": 5,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.5051,
        "sycophancy_rate": 1.0,
        "progressive": 0.40937312237131984,
        "regressive": 0.5906268776286802,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.5291735442976189
      },
      {
        "mode": "flat",
        "seed": 6,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.50375,
        "sycophancy_rate": 1.0,
        "progressive": 0.39656912209889,
        "regressive": 0.6034308779011099,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.5600516529849057
      },
      {
        "mode": "flat",
        "seed": 7,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.5046,
        "sycophancy_rate": 1.0,
        "progressive": 0.4029101856497742,
        "regressive": 0.5970898143502258,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.5163778993606631
      },
      {
        "mode": "flat",
        "seed": 8,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.505,
        "sycophancy_rate": 1.0,
        "progressive": 0.4064567876478845,
        "regressive": 0.5935432123521155,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.5220935488581043
      },
      {
        "mode": "flat",
        "seed": 9,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.50035,
        "sycophancy_rate": 1.0,
        "progressive": 0.39547108774767487,
        "regressive": 0.6045289122523251,
        "quartile_sycophancy": [
          1.0,
          1.0,
          1.0,
          1.0
        ],
        "coef_confidence": 0.4977066061082984
      }
    ],
    "confmod": [
      {
        "mode": "confmod",
        "seed": 0,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.547,
        "sycophancy_rate": 0.72879164594729,
        "progressive": 0.3188463451019393,
        "regressive": 0.4099453008453506,
        "quartile_sycophancy": [
          0.14797136038186157,
          0.7672105053720653,
          1.0,
          1.0
        ],
        "coef_confidence": 6.193794130405908
      },
      {
        "mode": "confmod",
        "seed": 1,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.5473,
        "sycophancy_rate": 0.7223054639896114,
        "progressive": 0.30935970432524224,
        "regressive": 0.4129457596643692,
        "quartile_sycophancy": [
          0.1346384338793448,
```

```
          0.7545963229416467,
          1.0,
          1.0
        ],
        "coef_confidence": 6.473871727258068
      },
      {
        "mode": "confmod",
        "seed": 2,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.5363,
        "sycophancy_rate": 0.7343325621164872,
        "progressive": 0.30852026959058443,
        "regressive": 0.425812292525902 8,
        "quartile_sycophancy": [
          0.1557344064386318,
          0.7814889336016096,
          1.0,
          1.0
        ],
        "coef_confidence": 6.30492430601265
      },
      {
        "mode": "confmod",
        "seed": 3,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.54985,
        "sycophancy_rate": 0.727826941986234,
        "progressive": 0.3177974434611603,
        "regressive": 0.41002949852507375,
        "quartile_sycophancy": [
          0.12780180888714118,
          0.7836349331235248,
          1.0,
          1.0
        ],
        "coef_confidence": 6.265967971716307
      },
      {
        "mode": "confmod",
        "seed": 4,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.5424,
        "sycophancy_rate": 0.7319597989949749,
        "progressive": 0.3085427135678392,
        "regressive": 0.4234170854271357,
        "quartile_sycophancy": [
          0.14630225080385853,
          0.781664656212304,
          1.0,
          1.0
        ],
        "coef_confidence": 6.364849641665376
      },
      {
        "mode": "confmod",
        "seed": 5,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.54365,
        "sycophancy_rate": 0.7351291808531945,
        "progressive": 0.31554175846184657,
        "regressive": 0.4195874223913479,
        "quartile_sycophancy": [
          0.1565879054865839,
          0.7840544871794872,
          1.0,
          1.0
        ],
        "coef_confidence": 6.633645519816753
      },
      {
        "mode": "confmod",
        "seed": 6,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.54445,
        "sycophancy_rate": 0.730978809283552,
        "progressive": 0.3031281533804238,
        "regressive": 0.42785065590312815,
        "quartile_sycophancy": [
          0.14769975786924938,
          0.7763423496164715,
          1.0,
          1.0
        ],
        "coef_confidence": 6.368821645240262
      },
      {
        "mode": "confmod",
        "seed": 7,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.54395,
        "sycophancy_rate": 0.736979427997993,
        "progressive": 0.3108881083793276,
        "regressive": 0.4260913196186653,
        "quartile_sycophancy": [
          0.14130871136089923,
          0.8065034122842232,
          1.0,
          1.0
        ],
        "coef_confidence": 6.2977841539167345
      },
      {
        "mode": "confmod",
        "seed": 8,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.54975,
        "sycophancy_rate": 0.7281933025867254,
        "progressive": 0.31542009223982354,
        "regressive": 0.41277321034690195,
        "quartile_sycophancy": [
          0.13753007217321572,
          0.7753710389089451,
          1.0,
          1.0
        ],
        "coef_confidence": 6.444369734590455
      },
      {
        "mode": "confmod",
        "seed": 9,
        "interp": "linear",
        "reward_noise": 0.0,
        "accuracy": 0.54785,
        "sycophancy_rate": 0.7238172260412454,
        "progressive": 0.3053983016579054,
        "regressive": 0.41841892438334005,
        "quartile_sycophancy": [
          0.1362717347351395,
          0.7589971694298423,
          1.0,
          1.0
        ],
        "coef_confidence": 6.394927698252903
      }
    ]
  },
  "interp_forms": {
    "linear": {
      "mode": "confmod",
      "seed": 0,
      "interp": "linear",
      "reward_noise": 0.0,
      "accuracy": 0.547,
      "sycophancy_rate": 0.72879164594729,
      "progressive": 0.3188463451019393,
      "regressive": 0.4099453008453506,
      "quartile_sycophancy": [
        0.14797136038186157,
        0.7672105053720653,
        1.0,
        1.0
      ],
      "coef_confidence": 6.193794130405908
    },
    "quadratic": {
      "mode": "confmod",
      "seed": 0,
      "interp": "quadratic",
      "reward_noise": 0.0,
      "accuracy": 0.5657,
      "sycophancy_rate": 0.6197911486822476,
      "progressive": 0.28294380905022376,
      "regressive": 0.33684733963202385,
      "quartile_sycophancy": [
        0.09188544152744631,
        0.5005968961400716,
        0.8866348448687351,
        1.0
      ],
      "coef_confidence": 3.7217937201439146
    },
    "sqrt": {
      "mode": "confmod",
      "seed": 0,
      "interp": "sqrt",
      "reward_noise": 0.0,
      "accuracy": 0.5252,
      "sycophancy_rate": 0.8761810044753854,
      "progressive": 0.3708602685231228,
      "regressive": 0.5053207359522626,
      "quartile_sycophancy": [
        0.5047732696897375,
        1.0,
        1.0,
        1.0
      ],
      "coef_confidence": 11.689379920156137
    },
    "step": {
      "mode": "confmod",
      "seed": 0,
      "interp": "step",
      "reward_noise": 0.0,
      "accuracy": 0.547,
      "sycophancy_rate": 0.72879164594729,
      "progressive": 0.3188463451019393,
      "regressive": 0.4099453008453506,
      "quartile_sycophancy": [
        0.14797136038186157,
        0.7672105053720653,
        1.0,
        1.0
      ],
      "coef_confidence": 6.193794130405908
    }
  },
  "noise_robustness": {
    "0.0": {
      "mode": "confmod",
      "seed": 0,
      "interp": "linear",
      "reward_noise": 0.0,
      "accuracy": 0.547,
      "sycophancy_rate": 0.72879164594729,
      "progressive": 0.3188463451019393,
      "regressive": 0.4099453008453506,
      "quartile_sycophancy": [
        0.14797136038186157,
        0.7672105053720653,
        1.0,
        1.0
      ],
      "coef_confidence": 6.193794130405908
    },
    "0.1": {
      "mode": "confmod",
      "seed": 0,
      "interp": "linear",
      "reward_noise": 0.1,
      "accuracy": 0.5229,
      "sycophancy_rate": 0.9358528095474888,
      "progressive": 0.3984087518647439,
      "regressive": 0.537444057682745,
      "quartile_sycophancy": [
        0.7434367541766109,
        1.0,
        1.0,
        1.0
      ],
      "coef_confidence": 1.9586824282617041
    },
    "0.25": {
      "mode": "confmod",
      "seed": 0,
      "interp": "linear",
      "reward_noise": 0.25,
      "accuracy": 0.5164,
      "sycophancy_rate": 0.9603182496270513,
      "progressive": 0.40417702635504726,
      "regressive": 0.556141223272004,
      "quartile_sycophancy": [
        0.8412887828162291,
        1.0,
        1.0,
        1.0
      ],
      "coef_confidence": 1.7370252223224019
    },
    "0.5": {
      "mode": "confmod",
      "seed": 0,
      "interp": "linear",
      "reward_noise": 0.5,
      "accuracy": 0.5085,
      "sycophancy_rate": 0.9869716558925907,
      "progressive": 0.40964694181999006,
      "regressive": 0.5773247140726007,
      "quartile_sycophancy": [
        0.9478918058870326,
        1.0,
        1.0,
        1.0
      ],
      "coef_confidence": 1.194160715836512
    }
  },
  "flat_label_diagnostic": {
    "min": 0,
    "max": 1,
    "mean": 0.500025
  },
  "step_threshold_shift": {
    "0.3": {
      "mode": "confmod",
      "seed": 0,
      "interp": "step",
      "reward_noise": 0.0,
      "accuracy": 0.5298,
      "sycophancy_rate": 0.8461461959224267,
      "progressive": 0.3604177026355047,
      "regressive": 0.4857284932869219,
      "quartile_sycophancy": [
        0.38862370723945905,
        0.9960206923995225,
        1.0,
        1.0
      ],
      "coef_confidence": 10.127868528668536
    },
    "0.5": {
      "mode": "confmod",
      "seed": 0,
      "interp": "step",
      "reward_noise": 0.0,
      "accuracy": 0.547,
      "sycophancy_rate": 0.72879164594729,
      "progressive": 0.3188463451019393,
      "regressive": 0.4099453008453506,
      "quartile_sycophancy": [
        0.14797136038186157,
        0.7672105053720653,
        1.0,
        1.0
      ],
      "coef_confidence": 6.193794130405908
    },
    "0.7": {
      "mode": "confmod",
      "seed": 0,
      "interp": "step",
      "reward_noise": 0.0,
      "accuracy": 0.56495,
      "sycophancy_rate": 0.6236698160119344,
      "progressive": 0.28413724515166583,
      "regressive": 0.3395325708602685,
      "quartile_sycophancy": [
        0.09148766905330151,
        0.5073617190608835,
        0.8957836117740652,
        1.0
      ],
      "coef_confidence": 3.808036378370021
    }
  },
  "sigmoid_steepness_shift": {
    "1.0": {
      "mode": "confmod",
      "seed": 0,
      "interp": "sigmoid",
      "reward_noise": 0.0,
      "accuracy": 0.547,
      "sycophancy_rate": 0.72879164594729,
      "progressive": 0.3188463451019393,
      "regressive": 0.4099453008453506,
      "quartile_sycophancy": [
        0.14797136038186157,
        0.7672105053720653,
        1.0,
        1.0
      ],
      "coef_confidence": 6.193794130405908
    },
    "5.0": {
      "mode": "confmod",
      "seed": 0,
      "interp": "sigmoid",
      "reward_noise": 0.0,
      "accuracy": 0.547,
      "sycophancy_rate": 0.72879164594729,
      "progressive": 0.3188463451019393,
      "regressive": 0.4099453008453506,
      "quartile_sycophancy": [
        0.14797136038186157,
        0.7672105053720653,
        1.0,
        1.0
      ],
      "coef_confidence": 6.193794130405908
    },
    "10.0": {
      "mode": "confmod",
      "seed": 0,
      "interp": "sigmoid",
      "reward_noise": 0.0,
      "accuracy": 0.547,
      "sycophancy_rate": 0.72879164594729,
      "progressive": 0.3188463451019393,
      "regressive": 0.4099453008453506,
      "quartile_sycophancy": [
        0.14797136038186157,
        0.7672105053720653,
        1.0,
        1.0
      ],
      "coef_confidence": 6.193794130405908
    },
    "30.0": {
      "mode": "confmod",
      "seed": 0,
      "interp": "sigmoid",
      "reward_noise": 0.0,
      "accuracy": 0.547,
      "sycophancy_rate": 0.72879164594729,
      "progressive": 0.3188463451019393,
      "regressive": 0.4099453008453506,
      "quartile_sycophancy": [
        0.14797136038186157,
        0.7672105053720653,
        1.0,
        1.0
      ],
      "coef_confidence": 6.193794130405908
    }
  }
}
```